

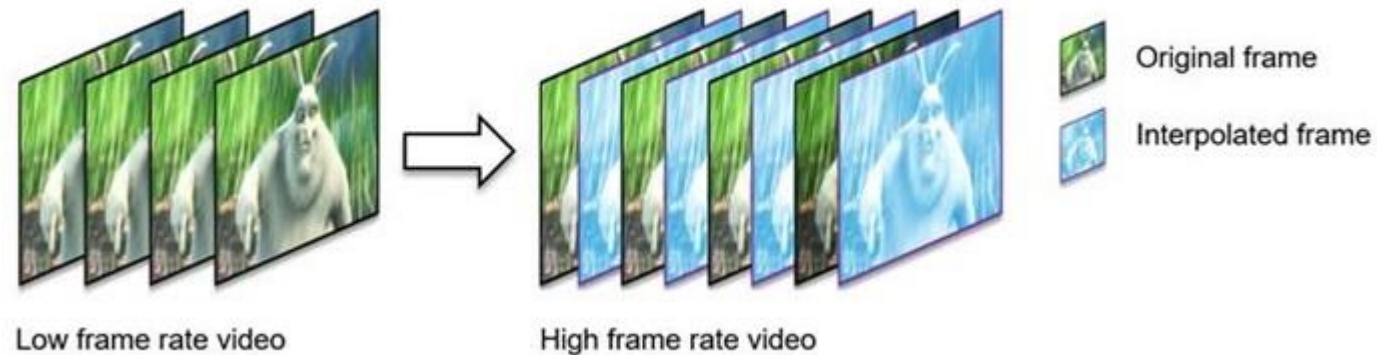
MEDIATEK

Computer Vision Final Project: Frame Generation

徐志瑋 cw.hsu@mediatek.com

Frame Generation

- ❑ Frame generation is a widely used technology in gaming and video processing to create and insert new frames between the existing frames.
 - 30 fps → 60 fps → 120 fps
- ❑ Frame generation can also be applied to video compression.
 - A to-be-compressed frame can be predicted by a generated frame.



Project Goal

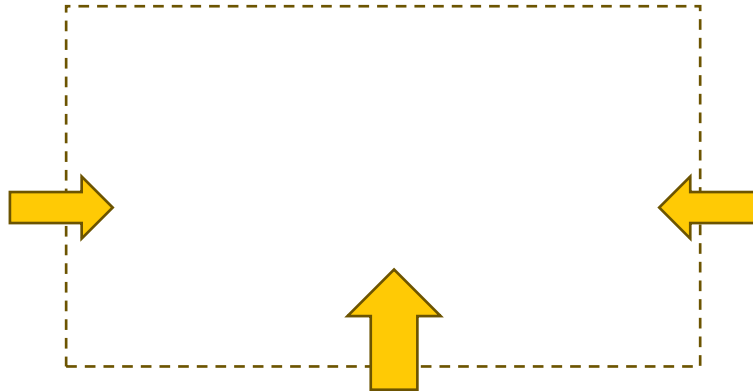
- ❑ Given base layer (BL) frame of time index t , enhancement layer (EL) frame of $t-1$, and EL frame of $t+1$, please generate EL frame of t
 - Given even frames of a 4K (3840x2160) video sequence
 - Given odd frames of a Full High-Definition (FHD, 1920x1080) video sequence
 - Generate odd frames of the 4K video sequence

$t = 0$



EL, 4K

$t = 1$



$t = 2$



BL, FHD



Project Goal

- ❑ Given base layer (BL) frame of time index t , enhancement layer (EL) frame of $t-1$, and EL frame of $t+1$, please generate EL frame of t
 - Given even frames of a 4K (3840x2160) video sequence
 - Given odd frames of a Full High-Definition (FHD, 1920x1080) video sequence
 - Generate odd frames of the 4K video sequence

$t = 0$



EL, 4K

$t = 1$



$t = 2$



BL, FHD



What You Need to Do

1. Download the data from TA

- Original 4K video sequences with odd frames
- Multi-layer bitstreams encoded by Versatile Video Coding (VVC/H.266), containing coded 4K layer and coded FHD layer
- VVC decoder
- Script for converting multi-layer bitstreams into even frames of 4K video sequences and odd frames of Full-HD video sequences
- PSNR and BD-rate calculating code

What You Need to Do

2. Convert multi-layer bitstreams into 4K/FHD video sequences as input

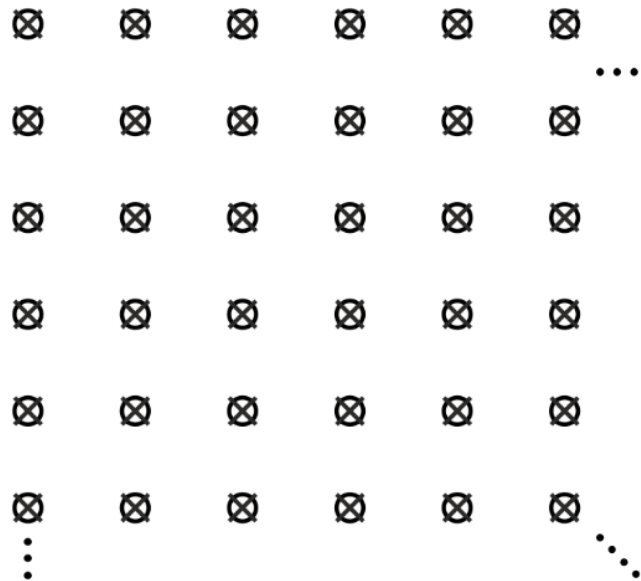
- For one original video sequence, there are 4 multi-layer bitstreams of different bitrates. Each bitstream will be decoded and converted into a 4K video sequence with even frames only and a FHD video sequence with odd frames only.
- The picture format is in 10-bit YUV420 progressive format.
 - One luma/chroma sample is stored in two bytes. The value is stored in the least significant 10 bits of the two bytes.



YUV420 Format

- **YUV444 format**

- One luma sample with one U sample and one V sample
- Can be transformed to RGB444 directly

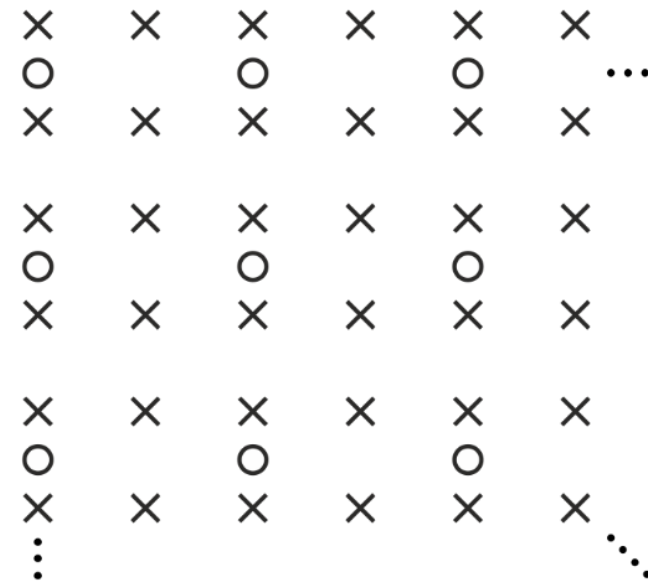


× Location of luma sample
○ Location of chroma sample

H.266(20)_F03

- **YUV420 format**

- 4 luma samples with one U sample and one V sample
- Need upsampling for chroma samples before transforming to RGB444



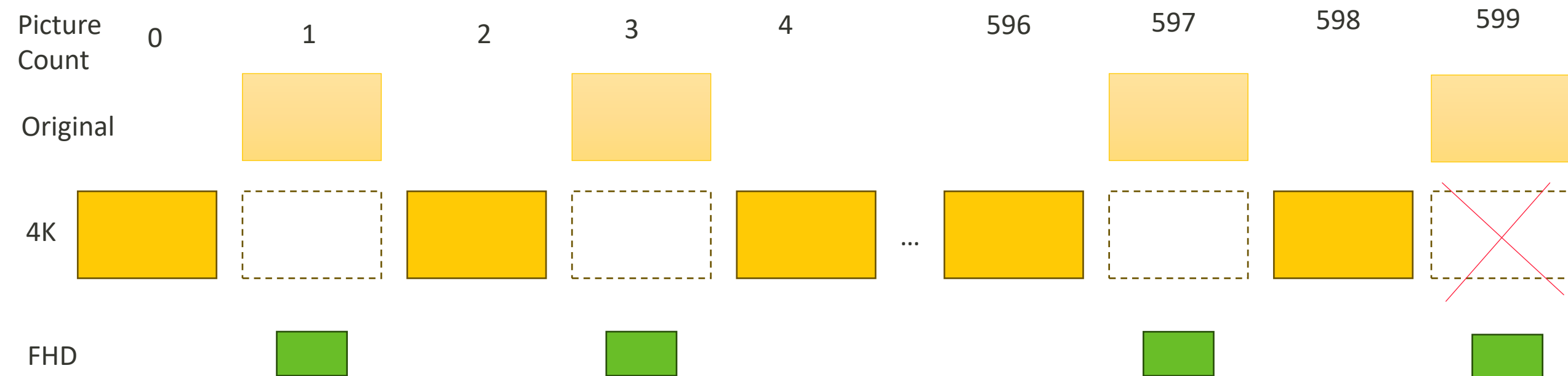
× Location of luma sample
○ Location of chroma sample

H.266(20)_F01

What You Need to Do

3. Generate 4K output video sequences with odd frames only

- You can use AI or non-AI methods
- Constraint: less than 200K MACs/pixel (3840x2160x200K MACs for a picture)
- Don't need to output the last frame



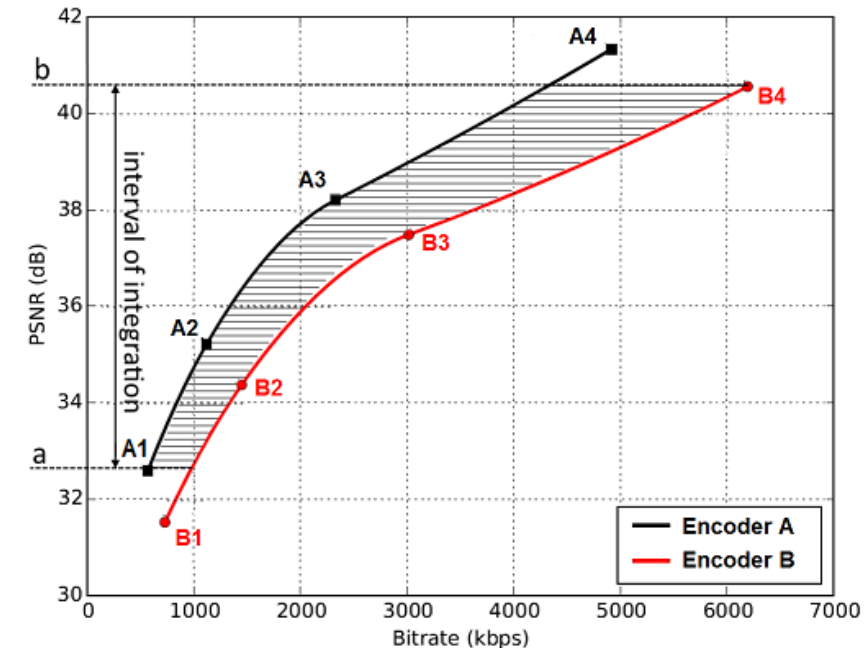
What You Need to Do

4. Calculate PSNR with original video sequences by using the provided code

$$PSNR = 10 \times \log_{10} \left(\frac{1020}{MSE} \right) \quad MSE = \frac{\sum (Ori - Pred)^2}{Total\ Samples}$$

5. Calculate BD-rate by using the provided code

- Need PSNR values of the 4 different bitrates
- Anchor: 4K video sequence upsampled from FHD video sequence by bicubic filter
- Negative value of BD-rate means better performance



Evaluation

- ❑ We provided 4 video sequences for development and pre-evaluation.
- ❑ Before the software deadline, please submit your source codes and report the BD-rates of the 4 video sequences.
- ❑ We will provide 2 hidden video sequences after the software deadline.
- ❑ Please report the BD-rates of the 4+2 video sequences.
 - Please use the same source codes before and after the software deadline.

Evaluation - Score Criterion

- ❑ **Objective quality: (50%)**
 - BD-rate ranking (of 4+2 video sequences)
 - In case of constraint violation
 - Complexity > 200K MACs/pixel (-25%)
 - Copy golden data directly (-50%)
- ❑ **Presentation: (50%) Top 10 teams**
 - Novelty and technical contribution (20%)
 - Experiment completeness (25%)
 - Presentation (5%)
- ❑ **Report: (50%) Other teams**
 - Novelty and technical contribution (25%)
 - Experiment completeness (25%)

Points	# of teams
50%	1
48%	2
46%	2
42%	The rest teams /4
38%	The rest teams /4
34%	The rest teams /4
30%	The rest teams /4

Evaluation Server

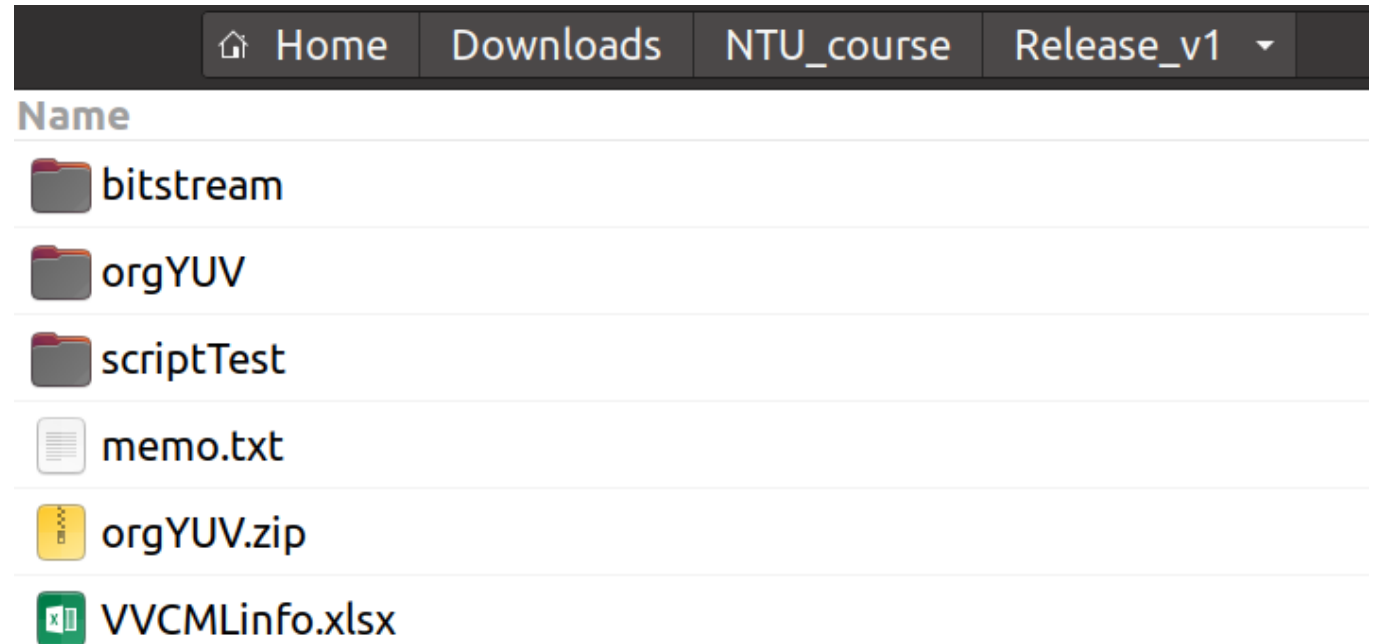
❑ To be announced by TA

Project Output

1. The report of your final project
2. Generated video sequences
3. Description of your method
4. Source codes

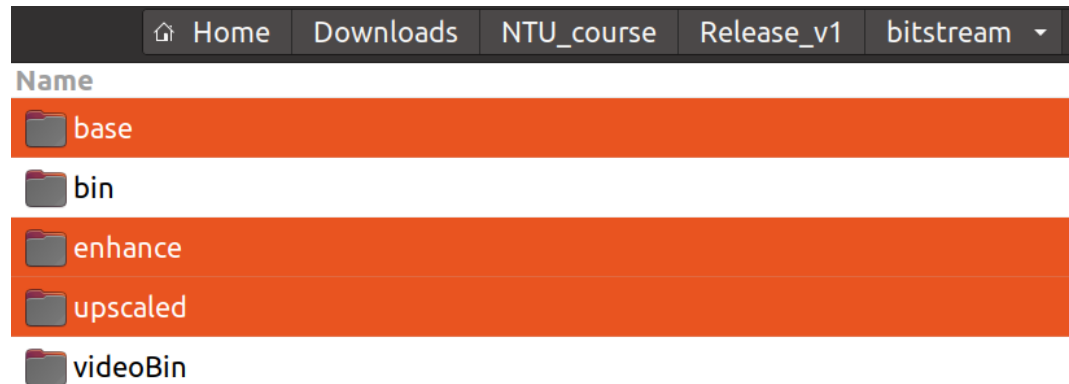
Initial final project directory

- ❑ **bitstream**: compressed video files to be decoded
- ❑ **orgYUV**: unzip orgYUV.zip. 4 uncompressed 4K videos with only odd frames
 - For PSNR calculation
 - View videos with YUView
 - <https://github.com/IENT/YUView>
- ❑ **scriptTest**
 - Scripts for calculating BD-rate
- ❑ **memo.txt**
 - Best BD-rate you can get!
- ❑ **VVCMLinfo.xlsx**
 - Video information

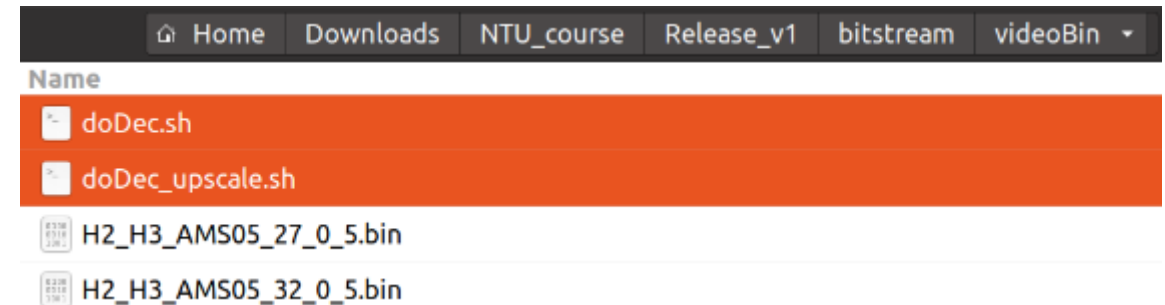


Decompress the bitstreams to get videos

- ❑ Make sure you get storage space ~300GB and make sure “ffmpeg” is installed (“ffmpeg” command available in terminal)
- ❑ Make sure there are 3 empty directories: “base/enhance/upscaled”



- ❑ @Ubuntu (>=20.04), go to “bitstream/videoBin”, run two scripts:
 - doDec.sh → get video in “base/enhance”
 - doDec_upscale.sh → get video in “upscaled”



Video decoding in progress

```
mtkuser@LNPC23110355: ~/Downloads/NTU_course/Release_v1/bitstream/videoBin
POC 364 LId: 0 TId: 3 ( RASL, B-SLICE, QP 30 ) [DT 0.056] [L0 360 356 352] [L1 368c 384] [:(,unk)]
POC 362 LId: 0 TId: 4 ( RASL, B-SLICE, QP 31 ) [DT 0.051] [L0 360 358 356 352] [L1 364c 368 384] [:(,unk)]
POC 361 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.035] [L0 360 356] [L1 362c 364] [:(,unk)]
POC 363 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.038] [L0 362c 360] [L1 364 368] [:(,unk)]
POC 366 LId: 0 TId: 4 ( RASL, B-SLICE, QP 31 ) [DT 0.048] [L0 364c 362 360 352] [L1 368 384] [:(,unk)]
POC 365 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.038] [L0 364 360] [L1 366c 368] [:(,unk)]
POC 367 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.035] [L0 366c 364] [L1 368 384] [:(,unk)]
POC 376 LId: 0 TId: 2 ( RASL, B-SLICE, QP 26 ) [DT 0.071] [L0 368c 360 352] [L1 384] [:(,unk)]
POC 372 LId: 0 TId: 3 ( RASL, B-SLICE, QP 30 ) [DT 0.058] [L0 368 360 352] [L1 376c 384] [:(,unk)]
POC 370 LId: 0 TId: 4 ( RASL, B-SLICE, QP 31 ) [DT 0.049] [L0 368 360 352] [L1 372c 376 384] [:(,unk)]
POC 369 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.036] [L0 368 360] [L1 370c 372] [:(,unk)]
POC 371 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.034] [L0 370c 368] [L1 372 376] [:(,unk)]
POC 374 LId: 0 TId: 4 ( RASL, B-SLICE, QP 31 ) [DT 0.047] [L0 372c 368 352] [L1 376 384 370] [:(,unk)]
POC 373 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.035] [L0 372 368] [L1 374c 376] [:(,unk)]
POC 375 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.033] [L0 374c 372] [L1 376 384] [:(,unk)]
POC 380 LId: 0 TId: 3 ( RASL, B-SLICE, QP 30 ) [DT 0.054] [L0 376c 372 368 352] [L1 384] [:(,unk)]
POC 378 LId: 0 TId: 4 ( RASL, B-SLICE, QP 31 ) [DT 0.046] [L0 376 372 368 352] [L1 380c 384] [:(,unk)]
POC 377 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.035] [L0 376 372] [L1 378c 380] [:(,unk)]
POC 379 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.034] [L0 378c 376] [L1 380 384] [:(,unk)]
POC 382 LId: 0 TId: 4 ( RASL, B-SLICE, QP 31 ) [DT 0.046] [L0 380c 376 368 352] [L1 384] [:(,unk)]
POC 381 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.034] [L0 380 376] [L1 382c 384] [:(,unk)]
POC 383 LId: 0 TId: 5 ( RASL, B-SLICE, QP 33 ) [DT 0.034] [L0 382c 380] [L1 384] [:(,unk)]
POC 416 LId: 0 TId: 0 ( TRAIL, B-SLICE, QP 24 ) [DT 0.089] [L0 384] [L1 384c] [:(,unk)]
POC 400 LId: 0 TId: 1 ( STSA, B-SLICE, QP 26 ) [DT 0.072] [L0 384 416] [L1 416c] [:(,unk)]
POC 392 LId: 0 TId: 2 ( TRAIL, B-SLICE, QP 26 ) [DT 0.069] [L0 384 400 416] [L1 400c 416] [:(,unk)]
POC 388 LId: 0 TId: 3 ( TRAIL, B-SLICE, QP 30 ) [DT 0.052] [L0 384 392 400] [L1 392c 400 416] [:(,unk)]
POC 386 LId: 0 TId: 4 ( TRAIL, B-SLICE, QP 31 ) [DT 0.045] [L0 384 388 392] [L1 388c 392 400 416] [:(,unk)]
POC 385 LId: 0 TId: 5 ( TRAIL, B-SLICE, QP 33 ) [DT 0.033] [L0 384] [L1 386c 388] [:(,unk)]
POC 387 LId: 0 TId: 5 ( TRAIL, B-SLICE, QP 33 ) [DT 0.032] [L0 386c 384] [L1 388 392] [:(,unk)]
```


Measuring your performance

- ❑ Go to “scriptTest”, there are 4 scripts for each video
 - testOddFrameProcession.py, testOddFrameAMS05.py, testOddFrameZombie.py, testOddFrameWalkInPark.py
 - For example, run “python testOddFrameZombie.py” to get the BD-rate
 - BD-rate = 0.000% means no improvement comparing to just upscaling the video (as the videos in “upscaled”)
 - More negative value of BD-rate means better performance (check last line of output to get BD-rate)

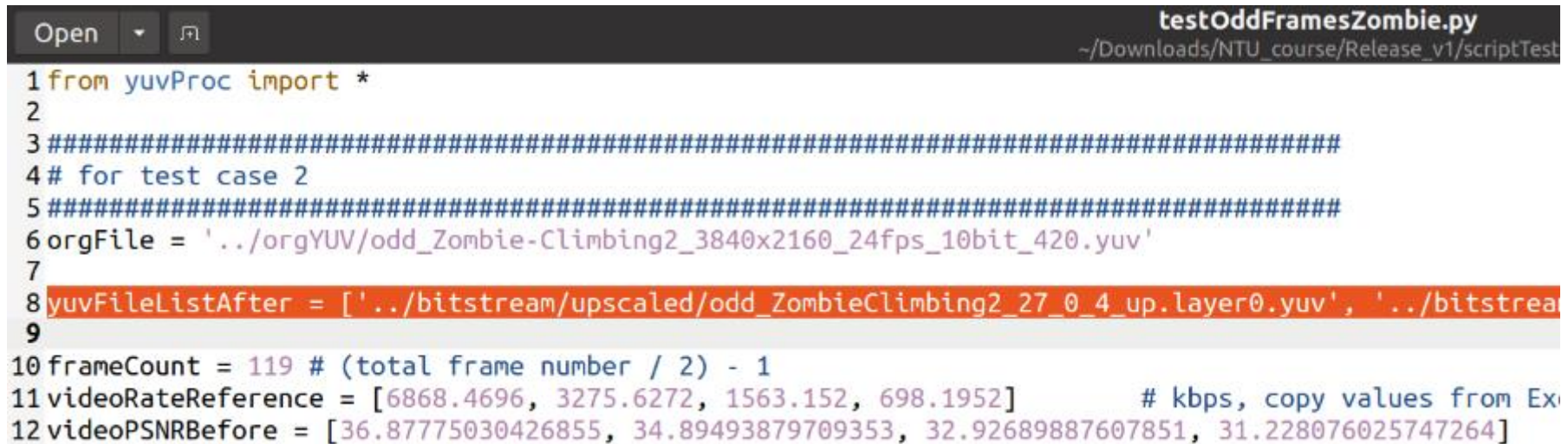
```
mtkuser@LNPC23110355:~/Downloads/NTU_course/Release_v1/scriptTest$ ls
__pycache__      testOddFramesProcession.py  testOddFramesZombie.py
testOddFramesAMS05.py  testOddFramesWalkInPark.py  yuvProc.py
mtkuser@LNPC23110355:~/Downloads/NTU_course/Release_v1/scriptTest$ python testOddFramesZombie.py
case 1:
  BDrate should be 15.851%
  Calculated BDrate = 15.851%

  Fake PSNR improvement:
  Calculated BDrate = -6.492%

case 2:
  PSNR before: [36.87775030426855, 34.89493879709353, 32.92689887607851, 31.228076025747264]
  PSNR after : [36.87775030426855, 34.89493879709353, 32.92689887607851, 31.228076025747264]
  Calculated BDrate = 0.000%
```

Put your own video files

- ❑ Put your 4 files of odd frames of 4K resolution from high quality to low quality to be the list (“yuvFileListAfter”) in line 8 as the figure below and run the script to get BD-rate
- ❑ Current exemplary video files in “yuvFileListAfter” are from “upscaled” where videos are upscaled by simple FIR filter
 - Upscaled videos are served as baseline videos therefore the BD-rate is 0.000%

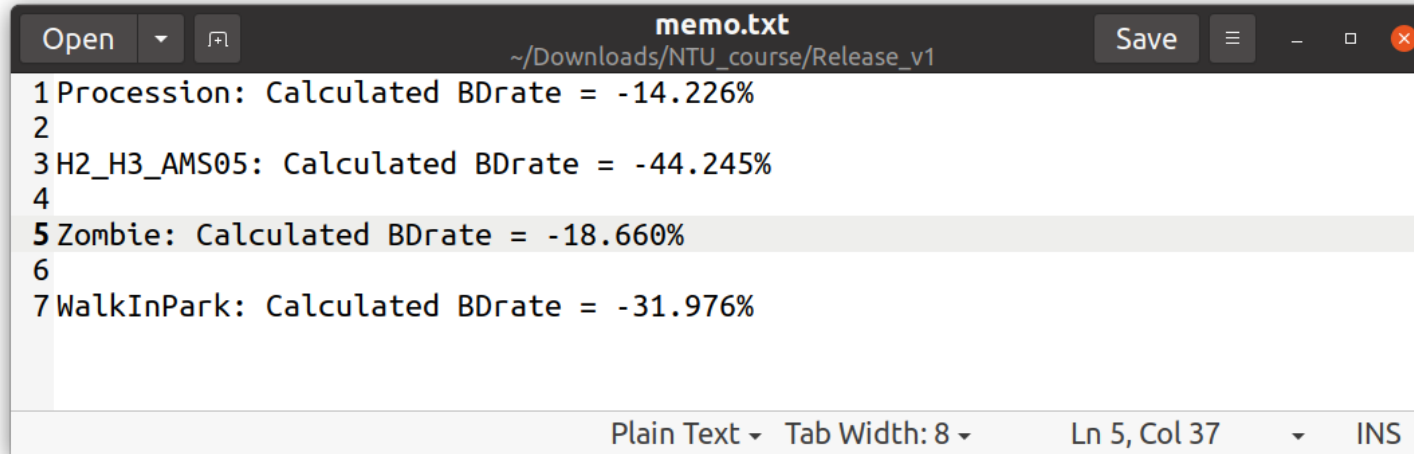


```
testOddFramesZombie.py
~/Downloads/NTU_course/Release_v1/scriptTest

1 from yuvProc import *
2
3 #####
4 # for test case 2
5 #####
6 orgFile = '../orgYUV/odd_Zombie-Climbing2_3840x2160_24fps_10bit_420.yuv'
7
8 yuvFileListAfter = ['../bitstream/upscaled/odd_ZombieClimbing2_27_0_4_up.layer0.yuv', '../bitstream/upscaled/odd_ZombieClimbing2_27_0_4_up.layer1.yuv', '../bitstream/upscaled/odd_ZombieClimbing2_27_0_4_up.layer2.yuv', '../bitstream/upscaled/odd_ZombieClimbing2_27_0_4_up.layer3.yuv']
9
10 frameCount = 119 # (total frame number / 2) - 1
11 videoRateReference = [6868.4696, 3275.6272, 1563.152, 698.1952] # kbps, copy values from Ex
12 videoPSNRBefore = [36.87775030426855, 34.89493879709353, 32.92689887607851, 31.228076025747264]
```

Best BD-rate you can get! Theoretically~~

- ❑ Best BD-rate is evaluated by the decoded odd frames of 4K resolution (EL)



```
memo.txt
~/Downloads/NTU_course/Release_v1

1 Procession: Calculated BDRate = -14.226%
2
3 H2_H3_AMS05: Calculated BDRate = -44.245%
4
5 Zombie: Calculated BDRate = -18.660%
6
7 WalkInPark: Calculated BDRate = -31.976%

Plain Text ▾ Tab Width: 8 ▾ Ln 5, Col 37 ▾ INS
```

- ❑ You should only use [1][2] as input
 - [1] Videos in “base”, BL, odd frames of FHD (1080p)
 - [2] Video in “enhance”, EL, even frames of 4K
 - To generate odd frames of 4K resolution
- ❑ Wish you a negative BD-rate! May the force be with you!

